One-Hot Encoding

What's Covered

1. Recap — Why Not Ordinal/Label Encoding for Nominal Data?

Ordinal Encoding and Label Encoding assign integers like 0, 1, 2 to categories. This works well when categories have a real order (School < Graduate < Postgraduate). But for nominal data — categories with NO natural order, like City or Color — assigning integers creates a false sense of rank that doesn't actually exist.

Example of the problem: If Mumbai=0, Delhi=1, Pune=2, a model might incorrectly learn that 'Pune is greater than Delhi is greater than Mumbai' — a meaningless and misleading relationship that was never present in the original data. One-Hot Encoding solves this.

2. What is One-Hot Encoding?

Definition: One-Hot Encoding converts a single categorical column into MULTIPLE new binary (0/1) columns — one column per unique category. For each row, exactly one of these new columns has the value 1 ('hot') and all the others have 0.

Because no integer ranking is involved, every category is treated as equally distinct from every other — exactly what nominal data requires.

2.1 Intuitive Analogy

Think of a row of light switches, one for each city: Mumbai, Delhi, Pune. For any single person, exactly ONE switch is flipped ON (1) — the city they belong to — and every other switch stays OFF (0). No switch is 'bigger' or 'smaller' than another; they simply represent independent yes/no facts.

3. Worked Example — Step by Step

Consider a 'City' column with three unique values across five rows.

3.1 Original Data

Row	City
1	Mumbai
2	Delhi
3	Pune
4	Mumbai
5	Delhi

3.2 After One-Hot Encoding

Row	City_Mumbai	City_Delhi	City_Pune
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0
5	0	1	0

> Notice: the single 'City' column became THREE new columns. Each row has exactly one 1, marking which city that row belongs to. No column is treated as numerically greater than another.

- A categorical column with k unique categories becomes k new binary columns after One-Hot Encoding.
- This is the main drawback: high-cardinality columns (many unique categories) create many new columns, increasing dimensionality.

4. One-Hot Encoding with pandas.get_dummies()

pandas provides a quick, convenient function for One-Hot Encoding — ideal for exploration, but less safe for production ML pipelines (explained in Section 5).

```python
# Import pandas
import pandas as pd

# Sample dataframe with a nominal categorical column
df = pd.DataFrame({'City': ['Mumbai', 'Delhi', 'Pune',
'Mumbai', 'Delhi']})

# One-Hot Encode the City column
df_encoded = pd.get_dummies(df, columns=['City'])

# Result has 3 new binary columns instead of 1 text column
print(df_encoded)
```

4.1 Useful Parameters

- prefix: customise the new column name prefix (default uses the original column name).
- dtype: control the output type — int (0/1) is most common for ML; default is sometimes bool.
- drop_first=True: drops the first category column to avoid redundancy (see Section 6).

> Limitation of get_dummies(): It has no fit/transform mechanism. If your test set has a category that wasn't in the training set (or is missing one that was), the columns won't match — leading to errors at prediction time. For production pipelines, prefer scikit-learn's OneHotEncoder.

5. OneHotEncoder in Scikit-learn

scikit-learn's OneHotEncoder follows the standard fit/transform pattern, ensuring the SAME columns are produced consistently for training and test data — critical for real pipelines.

```python
# Import the encoder
from sklearn.preprocessing import OneHotEncoder

# Create an instance — sparse_output=False returns a dense array
encoder = OneHotEncoder(sparse_output=False)

# Fit on training data — learns the unique categories
encoder.fit(X_train[['City']])

# Transform training and test data using the SAME categories
X_train_enc = encoder.transform(X_train[['City']])
X_test_enc = encoder.transform(X_test[['City']])

# View the generated column names
print(encoder.get_feature_names_out(['City']))
# e.g. ['City_Delhi' 'City_Mumbai' 'City_Pune']
```

5.1 Handling Unseen Categories

```python
# By default, an unseen category at transform time raises an error
# Use handle_unknown='ignore' to avoid this
encoder = OneHotEncoder(sparse_output=False,
handle_unknown='ignore')

# Unseen categories simply get all-zero columns instead of an error
encoder.fit(X_train[['City']])
X_test_enc = encoder.transform(X_test[['City']])
```

> CRITICAL: Always fit OneHotEncoder on training data only, then call transform() (not fit_transform()) on test data — the same data leakage rule applies as with every other encoder/scaler.

6. The Dummy Variable Trap — drop_first / drop='first'

If a column has k categories, only k-1 binary columns are needed to represent all the information. The k-th column is mathematically redundant because it can be perfectly predicted from the other k-1 columns (if all of them are 0, the dropped category must be 1).

6.1 Why It Matters

- Keeping all k columns introduces perfect multicollinearity — a problem specifically for linear models (Linear/Logistic Regression) where features are mathematically dependent on each other.
- This is called the Dummy Variable Trap.
- Tree-based models (Decision Trees, Random Forest, XGBoost) are NOT affected by this — they can handle redundant columns without issue.

6.2 Solution — Drop One Column

Python

```python
# Using pandas — drop_first removes the first category column
df_encoded = pd.get_dummies(df, columns=['City'],
drop_first=True)

# Using scikit-learn — drop='first' does the same thing
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(sparse_output=False, drop='first')
X_encoded = encoder.fit_transform(X_train[['City']])
```

Rule of Thumb: Use drop_first=True / drop='first' when feeding data into LINEAR models (Linear Regression, Logistic Regression, SVM). It is optional (but harmless) for tree-based models.

7. Handling High Cardinality Features

One-Hot Encoding works beautifully for columns with a small number of unique categories (3-10). But for columns with hundreds or thousands of unique values (e.g., Pincode, Product ID, User ID), it creates an enormous number of new sparse columns — leading straight back to the Curse of Dimensionality.

7.1 Strategies for High Cardinality

- Keep only the Top-N most frequent categories and group everything else into a single 'Other' category before encoding.
- Use Frequency Encoding: replace each category with how often it appears in the data (a single numeric column instead of many binary ones).
- Use Target Encoding: replace each category with the mean of the target variable for that category (requires care to avoid leakage).
- Use embeddings (in deep learning) to represent categories as dense, learned vectors.

7.2 Example — Grouping Rare Categories

Python

```
# Keep only the top 5 most frequent cities; group rest as 'Other'
top_cities = df['City'].value_counts().nlargest(5).index

df['City'] = df['City'].where(
df['City'].isin(top_cities), other='Other')

# Now one-hot encode — far fewer columns will be created
df_encoded = pd.get_dummies(df, columns=['City'])
```

8. Common Pitfalls

- ■ **Using One-Hot Encoding on ordinal data:** Wastes information about order. Use Ordinal Encoding instead when categories have a meaningful rank.
- ■ **Using One-Hot Encoding on high-cardinality columns:** Creates hundreds of sparse columns — group rare categories first or use an alternative encoding strategy.
- ■ **Forgetting drop_first for linear models:** Causes the Dummy Variable Trap (multicollinearity) — harmless for tree-based models, but harmful for linear ones.
- ■ **Using pandas.get_dummies() in a production ML pipeline:** No fit/transform mechanism — train and test sets can produce mismatched columns. Use sklearn's OneHotEncoder instead.
- ■ **Fitting the encoder on the full dataset before splitting:** Standard data leakage mistake — always fit on training data only.
- ■ **Not handling unseen categories:** Set handle_unknown='ignore' in OneHotEncoder to avoid errors on categories not seen during training.

9. Summary

Concept	Key Takeaway
What it does	Converts 1 categorical column into k binary columns (k = #categories)
Best for	Nominal data — categories with NO natural order
pandas method	pd.get_dummies(df, columns=[...]) — quick, no fit/transform
sklearn class	OneHotEncoder() — supports fit/transform, production-safe
handle_unknown	Set to 'ignore' to safely handle unseen categories at test time
Dummy Variable Trap	Keeping all k columns causes multicollinearity in linear models
Fix for the trap	drop_first=True (pandas) or drop='first' (sklearn)
Tree models	Not affected by the dummy variable trap — dropping is optional
High cardinality issue	Many unique categories -> too many new columns -> dimensionality curse
High cardinality fix	Group rare categories as 'Other', or use Frequency/Target Encoding
Fit rule	Always fit on training data only, transform on test data

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 27 — One-Hot Encoding